Quiuck dry cleaning company.

Assignment: Build a Mini Laundry Order Management System (AI-First)

Objective

Build a simple system that allows a dry cleaning store to:

- Create orders
- Track order status
- Calculate billing
- View basic dashboard data

You are expected to **use AI tools (ChatGPT, Copilot, etc.) heavily** and demonstrate how you leverage them.

Time Limit

72 hours

Problem Statement

A dry cleaning store needs a lightweight system to manage daily orders.

Core Features

1. Create Order

- Customer Name
- Phone Number
- Garments (e.g., Shirt, Pants, Saree)
- Quantity per garment
- Price per item (you can hardcode or configure)

👉 Output:

- Total bill amount
- Unique Order ID

---

2. Order Status Management

Each order should have statuses:

- RECEIVED
- PROCESSING
- READY
- DELIVERED

👉 Ability to update status

---

3. View Orders

- List all orders
- Filter by:
  - Status
  - Customer name / phone

---

4. Basic Dashboard

Return:

- Total orders
- Total revenue
- Orders per status

Tech Flexibility (Important)

You can use:

- Any language (Node.js, Python, etc.)
- Any framework (or none)
- Database OR in-memory storage

👉 **We care more about how you think and execute than your stack**

AI Usage Requirement (Critical)

You MUST:

- Use AI tools to build this
- Share:
    - Prompts you used
    - Where AI helped
    - Where you had to fix/improve AI code

Deliverables

1. Code Repository

- Public github link
- Clean structure

---

2. README (Very Important)

Include:

🔹 **Setup Instructions**

How to run your project

🔹 **Features Implemented**

What works

🔹 **AI Usage Report**

Explain:

- Which tools you used (ChatGPT, Copilot, etc.)
- Sample prompts
- What AI got wrong

- What you improved

🔹 **Tradeoffs**

- What you skipped
- What you'd improve with more time

---

3. API Collection / Demo

- Postman collection OR
- Simple UI OR
- Screenshots / Loom video

Bonus Tasks (Optional but High Weight)

- Add simple frontend (React / HTML)
- Add authentication (basic)
- Store data in DB (MongoDB / SQL)
- Add search (e.g., by garment type)
- Add “estimated delivery date”
- Deploy it (Render / Railway / AWS / Vercel)

What We Evaluate

We are NOT looking for perfect code.

We evaluate:

1. Speed & Execution

Did you build a working system quickly?

2. AI Leverage

Did you actually use AI effectively or just write manually?

3. Problem Solving

How you handled gaps, bugs, unclear requirements

4. Code Quality (Practical)

- Readable
- Logical
- Not over-engineered

5. Ownership Mindset

Did you go beyond minimum?

What Not to Do

- Don’t over-engineer
- Don’t spend time on fancy UI
- Don’t build unnecessary abstractions

Hint (What Good Candidates Do)

- Start with API quickly
- Use AI to scaffold
- Iterate fast
- Fix AI mistakes
- Keep it simple but working